

Rapport projet ARCHI

Ex 1 :

1.

J'ai factorisé les instructions `rrmovl` et `irmovl` sous le même icode, en conservant celui de **RRMOVL** (icode = 2). J'ai utilisé **ifun = 0** pour **rrmovl** et **ifun = 1** pour **irmovl**.

J'ai ensuite défini un nouveau symbole **RIMOVL** :

intsig RIMOVL 'instructionSet.get("rrmovl").icode

Puis, dans le code HCL, j'ai remplacé les anciennes occurrences de **RRMOVL** et **IRMOVL** par **((icode == RIMOVL) && (ifun == 0))** ou **((icode == RIMOVL) && (ifun == 1))**. Quand les deux instructions étaient traitées de la même façon, j'ai seulement utilisé **RIMOVL** (par exemple dans `int aluB`).

Dans `int aluA`, j'ai bien distingué les deux cas, car elles n'utilisent pas la même source de donnée :

- Pour **rrmovl** (ifun == 0), l'entrée est la valeur du registre source : `aluA = valA`
- Pour **irmovl** (ifun == 1), l'entrée est la valeur immédiate : `aluA = valC`

À l'étape **memory**, j'ai également distingué les cas:

- Pour `rrmovl`, il faut prendre le registre : `mem_addr = valA`, `mem_data = valB`
- Pour `irmovl`, il faut prendre la valeur immédiate : `mem_data = valC`

Ex 2 :

1.

- J'ai ajouté l'instruction **incl** dans l'éditeur du jeu d'instructions. Et, j'ai défini : **icode = 13** (comme pour `decl`, qui utilise aussi 13 mais avec un autre ifun) et **ifun = 0** (comme demandé). Elle prend l'argument **rA** car l'instruction **incl** agit sur un seul registre, sans constante immédiate.
- Tous les tests passent et sont reconnus par l'assembleur.

2.

- J'ai choisi d'utiliser **rA** comme registre cible pour l'instruction **incl %reg**. En effet, **rA** sert généralement à indiquer le registre source ou cible dans la plupart des instructions qui manipulent un seul registre comme **popl %rA**, **pushl %rA**. Par conséquent, ce choix respecte la logique et la cohérence avec les instructions existantes dans l'architecture Y86.
- De plus, cela limite des modifications du HCL aux blocs **srcA** et **dstE**, sans avoir à ajouter des cas particuliers ou des conditions supplémentaires pour gérer rB uniquement pour cette instruction.
- À l'inverse, rB est utilisé comme destination dans les instructions binaires telles que **addl %rA, %rB**. Donc, son utilisation pour une opération unaire comme **incl %reg** rendrait le HCL complexe et moins lisible, alors que l'utilisation de **rA** permet d'ajouter l'instruction de manière simple, cohérente et économique dans le code de contrôle.

3.

- Pour ajouter l'instruction **decl %reg**, j'ai choisi de factoriser au maximum le code de contrôle, en utilisant la même logique que pour l'instruction **incl %reg**.
- Plutôt que d'introduire un nouveau cas particulier dans la fonction de l'ALU comme l'utilisation la soustraction, j'ai décidé de profiter de la fonction d'addition (**ALUADD**) pour les deux instructions mais je différencie la valeur fournie à l'entrée aluB :
 - + Pour **incl**, j'utilise **aluA = valA** (la valeur du registre) et **aluB = 1**, ce qui donne **valA + 1**.
 - + Pour **decl**, j'utilise **aluA = valA** et **aluB = -1**, ce qui donne **valA + (-1)**, soit **valA - 1**.
- Donc, pour moi, cette solution offre à la fois simplicité et factorisation : il n'y a qu'un seul cas à gérer pour l'ALU et il suffit de changer la valeur d'entrée selon l'instruction. De surcroît, le comportement des instructions reste similaire, facile à comprendre et à maintenir puisqu'il n'est pas nécessaire de manipuler des permutations d'opérandes ou d'ajouter une gestion spécifique de la soustraction.
- Ainsi, la décrémentation est implémentée de façon élégante, conformément à la logique de l'architecture, et en minimisant la quantité de modifications nécessaires dans le code de contrôle.

4.

- J'ai écrit une fonction **main** qui teste **strncpy** dans deux cas :
 - + Premier test : la sentinelle 0 est rencontrée avant d'avoir copié n éléments.
 - + Deuxième test : on copie exactement n éléments sans rencontrer de sentinelle.

- Pour avancer les pointeurs source (**%esi**) et destination (**%edi**) de **4 octets** (taille d'un entier), j'ai utilisé **4 instructions incl** respectivement sur chacun. Chaque **incl** augmente l'adresse de 1 octet, donc 4 **incl** permettent d'avancer d'un entier. J'incrmente aussi **%eax** d'une unité pour suivre l'indice d'élément copié.

Ex 3 :

1.
 - J'ai ajouté l'instruction **LOOP** avec un nouveau **icode 14** et un **ifun** fixé à **0**. J'ai écrit un programme de test simple où **%ecx** est initialisé à 3, et dans chaque itération, **%eax** est incrémenté jusqu'à ce que **%ecx** atteigne 0.
 - La compilation fonctionne bien et reconnaît la nouvelle instruction.
2.
 - Au début, j'ai bien défini l'instruction **LOOP**, en définissant son icode, ses arguments et en modifiant le code HCL pour effectuer la décrémentation de **%ecx** et le saut vers le label lorsque **%ecx** n'atteint pas zéro.
 - Mais j'ai rencontré un problème. Toutefois, lors des tests, j'ai constaté que le programme ne sautait pas à la bonne adresse de boucle, mais revenait systématiquement au début du programme (adresse 0x0000). Cela provoque une boucle infinie du programme.
 - Après analyse, il semble que ce problème ne provient pas de la logique du processeur (HCL), mais j'ai rendu compte qu'il faut aussi l'ajouter dans **need_valC** sinon l'assembleur ne générerait pas d'adresse cible pour **LOOP**, et **valC** valait 0 par défaut.
 - Donc, j'ai corrigé cela en ajoutant **LOOP** dans **need_valC** qui le permet d'obtenir son adresse cible correctement, et la boucle fonctionne comme prévu.

Ex 4 :

1.

L'instruction **LOOP** utilise **icode = 14** et **ifun = 0** donc mes nouvelles instructions utilisent **icode = 14** et **ifun = 1** pour **LOOPE** et **ifun = 2** pour **LOOPNE**.

2.

Dans le code **HCL**, comme **LOOPE** et **LOOPNE** se comportent comme **LOOP**, j'ai ajouté ces instructions aux endroits où **LOOP** est déjà présenté : **bool need_valC**, **bool instr_valid**, **int srcB**, **int dstE**, **int aluB**.

Lors de l'exécution, l'**ALU** décrémente **%ecx** en utilisant **aluA = -1** et **aluB = valB**, avec une addition, sans mettre à jour les flags (**bool set_cc = icode in { OPL, IOPL }**; pas de loop dedans).

J'ai également intégré **LOOPE** et **LOOPNE** dans **is_bch**, pour que le saut soit décidé en fonction de **ifun** et des flags via **Bch**.

Pour le calcul du **new_pc**, si l'instruction est **LOOP**, **LOOPE** ou **LOOPNE**, et que **valE != 0**, alors on saute vers **valC** si c'est un **LOOP** simple ou si la condition de branchement est satisfaite : **(icode in { LOOP, LOOPE, LOOPNE } && valE != 0 && (icode == LOOP || Bch)) : valC;**

3.

Pour tester l'ajout de ces instructions, j'ai codé la fonction **strncpy**. La copie s'arrête soit lorsqu'on rencontre un zéro (sentinelle), soit lorsque le nombre maximal d'éléments à copier est atteint. La boucle utilise **LOOPNE**, combiné à un test de la valeur chargée pour mettre à jour **ZF**.

J'ai fait deux tests différents : un tableau contenant un zéro avant la fin, et un tableau sans zéro.